User management

Creating a `User` class in PHP involves implementing methods for user authentication, role-based authorization, and session management. Below is an outline of the `User` class, including methods for login, registration, role checking, and session handling. Additionally, I'll provide examples of corresponding HTML forms for user interaction.

PHP `User` Class

```
class User {
    private $pdo;
    private $userId;
    private $username;
    private $role;

    public function __construct($pdo) {
        $this->pdo = $pdo;
        session_start();
    }

    public function login($username, $password) {
        $stmt = $this->pdo->prepare("SELECT id, username, password, role FROM use
        $stmt->execute([$username]);
        $user = $stmt->fetch();

        if ($user && password_verify($password, $user['password'])) {
            $_SESSION['user_id'] = $user['id'];
            $_SESSION['username'] = $user['username'];
            $_SESSION['role'] = $user['role'];
            $this->userId = $user['id'];
            $this->username = $user['username'];
            $this->role = $user['role'];
            return true;
        }
        return false;
    }

    public function register($username, $password) {
        $hashedPassword = password_hash($password, PASSWORD_DEFAULT);
        $stmt = $this->pdo->prepare("INSERT INTO users (username, password) VALUE
        $stmt->execute([$username, $hashedPassword]);
    }

    public function logout() {
        session_unset();
        session_destroy();
```

```
    }

    public function isUserLoggedIn() {
        return isset($_SESSION['user_id']);
    }

    public function getUsername() {
        return $this->username;
    }

    public function getRole() {
        return $this->role;
    }

    public function isAdmin() {
        return $this->role === 'admin';
    }
}
```

Explanation

- **Constructor**: Initializes the database connection and starts the session.
- **Login**: Verifies user credentials and starts a session for the user.
- **Register**: Registers a new user with a hashed password.
- **Logout**: Ends the session.
- **Session Checks**: Methods like `isUserLoggedIn` and `isAdmin` check the user's login status and role.

HTML Forms

Registration Form (`register.html`)

```
<!DOCTYPE html>
<html>
<head>
    <title>Register</title>
</head>
<body>
    <h1>Register</h1>
    <form action="register.php" method="post">
        Username: <input type="text" name="username" required><br>
        Password: <input type="password" name="password" required><br>
        <input type="submit" value="Register">
    </form>
```

```
</body>
</html>
```

Login Form (`login.html`)

```
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
</head>
<body>
    <h1>Login</h1>
    <form action="login.php" method="post">
        Username: <input type="text" name="username" required><br>
        Password: <input type="password" name="password" required><br>
        <input type="submit" value="Login">
    </form>
</body>
</html>
```

PHP Scripts for Forms

Registration Script (`register.php`)

```
<?php
require_once 'User.php';
require_once 'db_connect.php'; // PDO connection

$user = new User($pdo);

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $user->register($_POST['username'], $_POST['password']);
    // Redirect to login or display a success message
}
```

Login Script (`login.php`)

```
<?php
require_once 'User.php';
require_once 'db_connect.php'; // PDO connection

$user = new User($pdo);

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    if ($user->login($_POST['username'], $_POST['password'])) {
        header("Location: dashboard.php"); // Redirect to a protected page
    } else {
```

```
            echo "Invalid credentials.";
        }
    }
```

Security Considerations

- Always sanitize and validate user inputs.
- Store hashed passwords in the database, never plain text.
- Implement CSRF protection on forms.
- Ensure secure session management, particularly on shared hosting environments.

This setup provides a foundational structure for user authentication and role-based authorization in your PHP application, integrating session management for maintaining user state across different pages.